

# ZYNXIS MOBILE APPLICATION

## FINAL CAPSTONE PROJECT DOCUMENTATION

**Prepared by:** Tayyeba Fatima

**Internship:** ZYNXIS Mobile Development Internship

**Project:** Zynxis Mobile Application

---

## 1. Project Overview

**Project Name:** Zynxis Mobile Application

**Project Type:** Flutter Mobile Application

**Development Framework:** Flutter

**Programming Language:** Dart

### Purpose

The Zynxis Mobile Application is a Flutter-based mobile application developed as part of the ZYNXIS Mobile Development Internship. The project demonstrates practical knowledge of mobile application development, user interface design, authentication, state management, API integration, persistent storage, and user profile management.

The main purpose of the application is to provide a functional and user-friendly mobile experience while demonstrating the concepts and technologies learned throughout the internship.

### Technologies Used

- **Flutter:** Used to develop the cross-platform mobile application.
- **Dart:** Used as the primary programming language.
- **Firebase Authentication:** Used for user registration and login functionality.
- **Provider:** Used for application state management.
- **SQLite:** Used for local persistent storage of favorite users.
- **SharedPreferences:** Used for storing lightweight application preferences and session-related data.
- **REST API:** Used to retrieve team directory information.
- **Android Studio:** Used for Flutter and Android development.
- **Visual Studio Code:** Used for source-code development and editing.

### Main Objectives

- Develop a functional and user-friendly mobile application.
- Implement multiple screens with proper navigation.
- Implement user authentication.
- Implement state management using Provider.
- Integrate an external REST API.
- Implement persistent local data storage.
- Implement light and dark theme functionality.
- Develop task management functionality.
- Implement favorites functionality.
- Create a user profile section.
- Apply responsive and reusable UI components.
- Combine all implemented features into a complete final application.

## **Application Structure**

The application contains multiple screens and modules, including:

- Splash Screen
- Login Screen
- Register Screen
- Dashboard
- Home
- About
- Services
- Tasks
- Favorites
- Profile
- Settings
- Notifications
- API / Team Directory

## **2. Application Features**

### **Authentication**

The application provides user registration and login functionality. Firebase Authentication is used to manage user authentication and provide secure access to the application.

### **Login**

The login screen allows users to enter their credentials and access the application. The authentication process validates the user and provides access to the dashboard after successful login.

### **Registration**

The registration screen allows new users to create an account and enter the application through the authentication system.

### **Dashboard**

The dashboard provides a central interface where users can access important application features and information. It provides navigation to different sections of the application.

### **Home Screen**

The Home screen provides the main application interface and displays important information and available application features.

### **About Screen**

The About screen provides information about the Zynxis application and its purpose.

### **Services Screen**

The Services screen displays the services and functionality provided by the application in an organized interface.

### **Task Management**

The application includes a task management feature that allows users to view and manage tasks. Task-related information is managed using application state.

### **Dark Mode**

The application supports both light and dark themes. Users can change the application theme according to their preference through the settings functionality.

### **API Integration**

The application integrates an external REST API to retrieve and display team directory information. API requests are handled through a dedicated API service, with error handling implemented using try-catch.

### **Navigation**

The application provides navigation between different screens, including Home, About, Services, Tasks, Favorites, Profile, Settings, and other application sections.

### **User-Friendly Interface**

The application uses reusable Flutter widgets and responsive layouts to provide a consistent and user-friendly interface across different screens and device sizes.

# 3. Advanced Features

## Provider State Management

Provider is used for state management throughout the application. Different providers are responsible for managing application features such as authentication, counter functionality, favorites, tasks, and theme management.

The application uses `ChangeNotifier`-based providers to maintain application state. The `notifyListeners()` method is used to notify listening widgets when the state changes, allowing the user interface to update automatically.

## Authentication State

The authentication provider manages the user's authentication state and provides information about the current user and login status.

## Theme State

The theme provider manages the application's light and dark theme settings. The selected theme can be maintained using persistent preferences.

## Favorites

The application provides a Favorites feature that allows users to save users to a favorites list. Saved users can be viewed through the Favorites screen.

## SQLite Persistence

SQLite is used for local persistent storage of favorite-user information. The database service manages the creation, insertion, retrieval, checking, and deletion of favorite records.

This allows favorite information to remain available when the user navigates between screens and returns to the application.

## Database Operations

- **Insert Favorite:** Adds a user to the favorites database.
- **Retrieve Favorites:** Fetches stored favorite users.
- **Check Favorite:** Checks whether a user already exists in the favorites database.
- **Delete Favorite:** Removes a selected favorite user.
- **Delete All Favorites:** Removes all stored favorite records.

## SharedPreferences

SharedPreferences is used for storing lightweight application preferences and session-related information. It helps maintain selected settings and user preferences between application sessions.

## **Profile**

The Profile screen provides a dedicated area for displaying user-related information. It allows user information to be presented in an organized and accessible manner.

## **Notifications and Settings**

The application includes notification and settings functionality. The Settings screen provides options for managing application preferences, including theme-related settings and other user preferences.

## **Responsive Design**

The application uses responsive layouts and reusable widgets to improve the user experience on different screen sizes.

## **Error Handling**

The application uses try-catch error handling for API operations. This helps prevent application crashes when an API request fails and allows the application to handle errors appropriately.

# 4. Conclusion

## Final Result

The Zynxis Mobile Application successfully combines multiple mobile development concepts into a functional Flutter application. The final application includes authentication, navigation, dashboard functionality, task management, API integration, state management, dark mode, favorites, persistent storage, profile functionality, notifications, and settings.

The project demonstrates the practical implementation of concepts learned during the ZYNXIS Mobile Development Internship.

## What I Learned

During the development of this project, I gained practical experience in Flutter and Dart development. I learned how to design mobile user interfaces, implement screen navigation, manage application state using Provider, integrate Firebase Authentication, consume REST APIs, handle API errors, use SQLite for local storage, use SharedPreferences for application preferences, and develop reusable Flutter widgets.

I also improved my understanding of project organization, debugging, application testing, and Flutter development workflows.

## Challenges Encountered

During development, I faced challenges related to Flutter project configuration, Android build configuration, state management, API integration, database persistence, navigation, responsive UI design, and debugging.

Resolving these challenges helped me improve my problem-solving and debugging skills and provided practical experience in identifying and fixing issues during mobile application development.

## Future Improvements

Future versions of the Zynxis application could include:

- Improved UI animations and transitions.
- Enhanced authentication and security features.
- Cloud synchronization of favorite data.
- Additional API functionality.
- Improved notification features.
- More user customization options.
- Enhanced task management functionality.
- Additional application features and services.

- Improved performance and optimization.
- Deployment to the Google Play Store.

### **Final Statement**

This project provided practical experience in developing a complete Flutter mobile application. It allowed me to apply the concepts learned throughout the internship and strengthened my understanding of modern mobile application development.

The Zynxis Mobile Application represents the completion of the internship project and demonstrates the practical use of Flutter, Dart, Firebase, Provider, REST APIs, SQLite, and persistent storage.

**Project Status:** Completed

**Prepared by:** Tayyeba Fatima

**ZYNXIS Mobile Development Internship — Final Capstone Project**

**THANK YOU**